COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | October 6, 2026 | **DATE SUBMITTED:** | October 7, 2026 |
| **NAME:** | Asajar, Carlo Mari<br>Godino, Jaime Jr. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

## ACTIVITY NO. 11.1 BASIC ALGORITHM ANALYSIS

### 1. PROCEDURE OUTPUT

**ILO A: Measure the runtime of algorithms using theoretical analysis**

Assess if two arrays have any common values.

Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: $O(n^2)$

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.

| | |
|---|---|
| Analysis | The graph shows that the ***number of operations increases faster*** when the input size becomes bigger. When the size is ***10***, the number of operations is ***110***, then it becomes ***420*** when the size is ***20***, and reaches ***930*** when the size is ***30***. This shows that the operations do not increase in a simple way because the growth is ***quadratic***. The graph becomes more curved as the size increases because more elements need to be checked during the search. It can be observed that the number of operations follows the formula $n^2 + n$, where the $n^2$ is the main part that causes the fast increase. Even though there is also an additional n operations, the overall worst-case complexity is still $O(n^2)$. This means that as the input becomes larger, the running time can increase a lot, making the algorithm less efficient for very large arrays. |

**ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis**

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

| First Name | Last Name | Flu Shot? |
|---|---|---|
| 1 | 3 | Yes |
| 2 | 2 | No |
| 2 | 3 | Yes |

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible.

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the Student class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL std::sort() function.
3. Use a binary search to see if the student is present on the list.
4. If the student isn't present in the list, your function should return true since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. 6. Else, return false.

Step 0: Begin by including the following headers:

**Code:**

PROCEDURE > main_Procedure_ILOB.cpp > Student

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
```

Step 1: Define the STUDENT class as follows:

**Code:**

```cpp
class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {
    }

    // Getters
    auto get_name() {
        return name;
    }

    auto is_vaccinated() {
        return vaccinated;
    }
```

Step 2: Overload the required operators for the Student class so that a vector of students can be sorted using the STL std::sort() function.

**Code:**

```
26      // Two people are same if they have the same name
27      bool operator==(const Student& p) const {
28          return this->name == p.name;
29      }
30
31      // The ordering of a set of people is defined by their name
32      bool operator<(const Student& p) const {
33          return this->name < p.name;
34      }
35
36      bool operator>(const Student& p) const {
37          return this->name > p.name;
38      }
39  };
```

Step 3: The following function lets us generate a student from random data:

**Code:**

```
41  auto generate_random_Student(int max) {
42      std::random_device rd;
43      std::mt19937 rand(rd());
44
45      // the IDs of Student should be in range [1, max]
46      std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);
47
48      // Generate random credentials
49      auto random_name = std::make_pair(
50          uniform_dist(rand),
51          uniform_dist(rand)
52      );
53
54      bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
55
56      return Student(random_name, is_vaccinated);
57  }
```

Step 4: For the next important function, we will implement the use of std::chrono for experimental analysis. The following code is used to run and test the output of our implementation.

**Code:**

```
90  void search_test(int size, Student p) {
91      std::vector<Student> people;
92
93      // Create a list of random people
94      for (auto i = 0; i < size; i++)
95          people.push_back(generate_random_Student(size));
96
97      std::sort(people.begin(), people.end());
98
99      // To measure the time taken, start the clock
100     std::chrono::steady_clock::time_point begin =
101         std::chrono::steady_clock::now();
102
103     bool search_result = needs_vaccination(p, people);
104
105     // Stop the clock
106     std::chrono::steady_clock::time_point end =
107         std::chrono::steady_clock::now();
108
```

```
109     std::cout << "Time taken to search = "
110               << std::chrono::duration_cast<std::chrono::microseconds>
111                  (end - begin).count()
112               << " microseconds" << std::endl;
113
114     if (search_result)
115         std::cout << "Student (" << p.get_name().first << " "
116                   << p.get_name().second << ") "
117                   << "needs vaccination." << std::endl;
118     else
119         std::cout << "Student (" << p.get_name().first << " "
120                   << p.get_name().second << ") "
121                   << "does not need vaccination." << std::endl;
122 }
```

Step 5: The following function implements our logic for whether a vaccination is needed:

**Code:**

```
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (first < last) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence
        // and he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;

        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;

        else if (mid_element > P)
            last = first + mid_element_index;

        else if (mid_element < P)
            first = first + mid_element_index + 1;

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }

    return true;
}
```

Step 6: Finally the driver code is implemented as follows:

**Code:**

```
124  int main() {
125      // Generate a Student to search
126      auto p = generate_random_Student(1000);
127
128      search_test(1000, p);
129      search_test(10000, p);
130      search_test(100000, p);
131
132      return 0;
133  }
```

Fill out the table.
**(Table 11-1. Results of Runtime Performance)**

| Input Size (N) | Execution Speed | Screenshot | Observation(s) |
|---|---|---|---|
| 1,000 | 1 microsecond | Time taken to search = 1 microseconds<br>Student (566 437) needs vaccination. | The search finished very fast with 1,000 inputs. Only a small number of comparisons were needed because the search keeps dividing the range in half. The execution time was low, showing that the search can handle this input size easily. |

| | | | |
|---|---|---|---|
| **10,000** | **0 microseconds** | `Time taken to search = 0 microseconds`<br>`Student (566 437) needs vaccination.` | With 10,000 inputs, the search was still fast even if the input size became bigger. The execution time only changed a little because the search does not check every element one by one. Instead, it keeps cutting the search range in half. |
| **100,000** | **8 microseconds** | `Time taken to search = 8 microseconds`<br>`Student (566 437) needs vaccination.` | With 100,000 inputs, the search still finished in a short time. Even though there were many more inputs, the execution time did not become extremely slow. This shows that the binary search can still work efficiently with a larger amount of sorted data. |

Table 11-1. Results of Runtime Performance

Provide your own analysis of the outcome listed in this table.

| | |
|---|---|
| Analysis | The results showed that the search stayed very fast even when the input size increased from ***1,000 to 100,000***. The execution time was ***1 microsecond for 1,000 inputs, 0 microseconds for 10,000 inputs, and 8 microseconds for 100,000 inputs***. The small changes in time can happen because of the computer's performance and other processes running while the program is being tested. The ***10,000 input*** showing ***0 microseconds*** does not mean that no searching happened, but the search was just too fast for the timer to record at least ***1 microsecond***. Even with ***100,000 inputs***, the search only took ***8 microseconds***. This happened because ***the binary search keeps dividing the search range in half instead of checking every element one by one***. Overall, the results showed that the search stayed efficient as the input became bigger, which ***matches the expected logarithmic runtime of O(log N)***. |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis**

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

**Problem 1:** Consider a program that returns true if the elements in an array are unique

Code

SUPPLEMENTARY > main_Supplementary_ILOA.cpp > brpower(long long, int)

```cpp
#include <iostream>
#include <vector>
#include <chrono>

// PROBLEM 1: UNIQUE
bool Unique(std::vector<int> A){

    int n = A.size();

    for(int i = 0; i < n - 1; i++){
        for(int j = i + 1; j < n; j++){
            if(A[i] == A[j]){
                return false;
            }
        }
    }

    return true;
}

// Experimental Analysis for UNIQUE
void testUnique(int size){

    std::vector<int> A;

    // Create an array with unique values
    for(int i = 0; i < size; i++){
        A.push_back(i);
    }

```

```cpp
    // Start the timer
    auto begin = std::chrono::steady_clock::now();
    bool result = Unique(A);

    // Stop the timer
    auto end = std::chrono::steady_clock::now();

    std::cout << "\nInput Size: " << size << std::endl;
    std::cout << "----------------------------------------" << std::endl;
    std::cout << "Execution Time: " << std::chrono::duration_cast<
                std::chrono::microseconds>(end - begin).count()
                << " microseconds" << std::endl;

    if(result){
        std::cout << "Result: All elements are unique." << std::endl;
    }
    else{
        std::cout << "Result: Duplicate element found." << std::endl;
    }
}
```

```cpp
int main(){

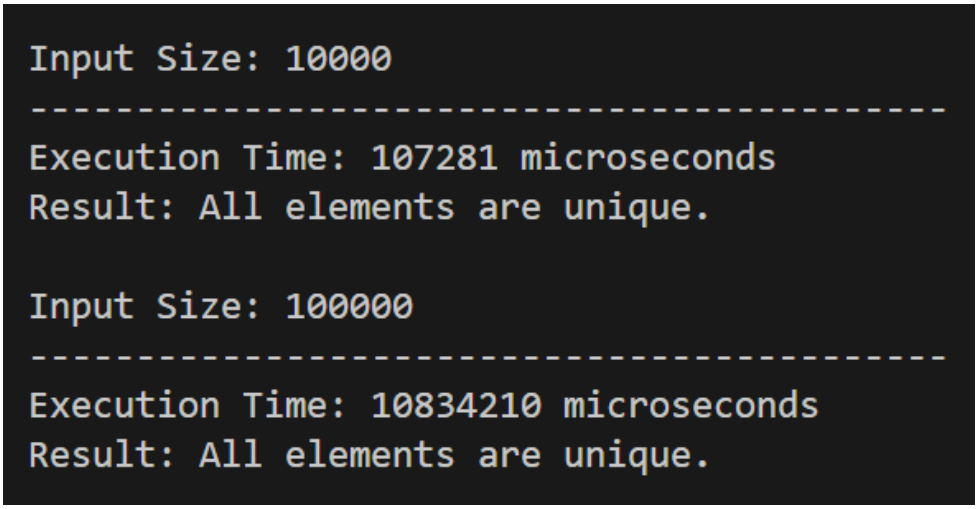
    // Problem 1
    std::cout << "\nPROBLEM 1: UNIQUE" << std::endl;
    std::cout << "----------------------------------------" << std::endl;

    testUnique(1000);
    testUnique(10000);
    testUnique(100000);
```

Output

```
PROBLEM 1: UNIQUE
----------------------------------------

Input Size: 1000
----------------------------------------
Execution Time: 1193 microseconds
Result: All elements are unique.
```

| | |
|---|---|
| |  |

| **Asajar's Analysis:** | |
|---|---|
| Theoretical Analysis | The ***Unique algorithm*** checks if all the elements in an array are unique by ***comparing each element with the other elements after it***. It uses ***two for loops***, where the first loop selects an element and the second loop compares it with the remaining elements. In the ***worst case***, all the elements are unique, so ***all possible comparisons*** need to be checked before returning true. The number of comparisons is based on $\frac{n(n-1)}{2}$, so the ***time complexity is O(n²)***. The space ***complexity is O(1)*** because only a few variables are used and no extra memory that grows with the input size is needed. |
| Experimental Analysis | The program was tested using ***input sizes of 1,000, 10,000,*** and ***100,000***. For 1,000 elements, the ***execution time was 1,193 microseconds***, and the result showed that ***all elements were unique***. For 10,000 elements, the ***execution time increased to 107,281 microseconds***, and the result was still that all elements were unique. For 100,000 elements, the ***execution time became 10,834,210 microseconds***, which is around ***10.83 seconds***, and all elements were still unique. The ***bigger input took much more time*** because the algorithm needed to ***compare many pairs of elements***. |
| Analysis and Comparison | The experimental result shows the same general behavior as the theoretical analysis of ***O(n²)***. When the input increased from 1,000 to 10,000, the input became ***10 times larger***, while the execution time increased from 1,193 microseconds to ***107,281 microseconds***. When the input reached 100,000, the execution time became ***10,834,210 microseconds or around 10.83*** |

| | |
|---|---|
| | ***seconds***. This shows that the algorithm becomes ***much slower when the input gets bigger*** because the two loops need to do a ***very large number of comparisons***. The actual result ***supports the theoretical idea that the algorithm has quadratic growth***. |

| **Godino's Analysis:** | |
|---|---|
| Theoretical Analysis | The Unique algorithm checks if all the elements in the array are different from each other. It uses two for loops, where the first loop selects an element and the second loop compares it with the elements after it. In the worst case, all the elements are unique, so every possible comparison needs to be checked before returning true. The number of comparisons is based on $\frac{n(n-1)}{2}$, so the time complexity is O(n²). The space complexity is O(1) because the algorithm does not need extra memory that increases with the input size. |
| Experimental Analysis | The program was tested using input sizes of 1,000, 10,000, and 100,000. For an input size of 1,000, the execution time was 1,193 microseconds, and the result showed that all elements were unique. When the input size increased to 10,000, the execution time became 107,281 microseconds, while the result was still that all elements were unique. For 100,000 elements, the execution time increased to 10,834,210 microseconds, which is around 10.83 seconds. The result was still the same because the array only contained unique values. |
| Analysis and Comparison | The experimental result shows the same general behavior as the theoretical analysis of O(n²). When the input size became bigger, the execution time also increased very quickly. From 1,000 to 10,000 elements, the input became 10 times larger, while the execution time increased from 1,193 microseconds to 107,281 microseconds. When the input reached 100,000, the execution time became 10,834,210 microseconds. This shows that the algorithm becomes much slower when the input becomes large because the two loops need to perform a very large number of comparisons. The actual result supports the theoretical idea that the algorithm has quadratic growth. |

**Problem 2:** Consider another program that raises a number x to an arbitrary nonnegative integer, n.

Code

```
// PROBLEM 2: RPOWER
long long rpower(long long x, int n){

    if(n == 0){
        return 1;
    }

    return x * rpower(x, n - 1);
}

// PROBLEM 2: BRPOWER
long long brpower(long long x, int n){

    if(n == 0){
        return 1;
    }

    if(n % 2 != 0){
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    long long y = brpower(x, n / 2);

    return y * y;
}

// Experimental Analysis for Power
void testPower(int size){

    std::cout << "\nInput Size: " << size << std::endl;
    std::cout << "----------------------------------------" << std::endl;

    // Test rpower
    auto begin1 = std::chrono::steady_clock::now();
    volatile long long result1 = rpower(2, size);
    auto end1 = std::chrono::steady_clock::now();

    std::cout << "rpower Execution Time: " << std::chrono::duration_cast<
                 std::chrono::microseconds>(end1 - begin1).count()
                 << " microseconds" << std::endl;

    // Test brpower
    auto begin2 = std::chrono::steady_clock::now();
    volatile long long result2 = brpower(2, size);
```

```
    auto end2 = std::chrono::steady_clock::now();

    std::cout << "brpower Execution Time: " << std::chrono::duration_cast<
                 std::chrono::microseconds>(end2 - begin2).count()
                 << " microseconds" << std::endl;
}
```

```
    // Problem 2
    std::cout << "\nPROBLEM 2: POWER" << std::endl;
    std::cout << "----------------------------------------" << std::endl;

    testPower(1000);
    testPower(10000);
    testPower(100000);

    return 0;
}
```

Output

```
PROBLEM 2: POWER
----------------------------------------

Input Size: 1000
----------------------------------------
rpower Execution Time: 37 microseconds
brpower Execution Time: 0 microseconds

Input Size: 10000
----------------------------------------
rpower Execution Time: 280 microseconds
brpower Execution Time: 0 microseconds

Input Size: 100000
----------------------------------------
```

**RPOWER**

| **Asajar's Analysis:** | |
|---|---|
| Theoretical Analysis | The ***rpower algorithm*** calculates the power by ***repeatedly calling itself*** and ***decreasing the value of n by one*** each time. For example, if n is 5, the calls go from 5 to 4, then 3, 2, 1, and finally 0. Since ***n only decreases by one in every recursive call***, a bigger value of n will need ***more calls***. Because of this, the ***time complexity is O(n)***. The ***space complexity is also O(n)*** because all the recursive calls need memory until the calls finish returning. |
| Experimental Analysis | The program was tested using ***input sizes of 1,000, 10,000,*** and ***100,000***. For 1,000, the ***execution time was 37 microseconds***. When the input became 10,000, the ***execution time increased to 280 microseconds***. When the input reached 100,000, the program showed the input size but did not show the execution time because the large number of recursive calls caused a ***segmentation fault***. This most likely happened because the recursion used ***too much stack memory.*** Because of this, there was ***no exact execution time recorded for the 100,000 input***. |
| Analysis and Comparison | The experimental result generally supports the theoretical analysis of ***O(n)***. From 1,000 to 10,000, the input became ***10 times larger*** and the execution time increased from ***37 microseconds to 280 microseconds***. This happens because ***rpower only decreases n by one in every recursive call***, so a bigger input needs ***many more calls***. At 100,000, the program got a ***segmentation fault*** before the execution time could be displayed. This also shows that the algorithm can have a problem with ***very large inputs*** because the recursive calls can use ***too much stack memory***. |

| **Godino's Analysis:** | |
|---|---|
| Theoretical Analysis | The rpower algorithm calculates the power by repeatedly calling itself and decreasing the value of n by one in every recursive call. For example, if the value of n is 5, the process goes from 5 to 4, then 3, 2, 1, and finally 0. Because the value only decreases by one each time, the number of recursive calls depends on the size of n. Because of this, the time complexity of rpower is O(n). Its space complexity is also O(n) because the recursive calls need memory while they are being processed. |
| Experimental Analysis | The program tested rpower using input sizes of 1,000, 10,000, and 100,000. For an input size of 1,000, the execution time was 37 microseconds. When the input size increased to 10,000, the execution time became 280 microseconds. The output for the 100,000 input size only shows the beginning of the test, and the execution time was not included because the program encountered a segmentation fault during the rpower calculation. Because of this, an exact result |

| | |
|---|---|
| | for 100,000 cannot be given. |
| Analysis and Comparison | The experimental results generally support the theoretical analysis of O(n). When the input size increased from 1,000 to 10,000, the execution time also increased from 37 microseconds to 280 microseconds. This happens because rpower decreases the value of n by only one for every recursive call. Because of this, a larger input requires more recursive calls. At 100,000, the program encountered a segmentation fault before the execution time could be displayed. The result still shows that the execution time increases as the input becomes larger, which agrees with the expected linear growth of the algorithm. |

**BRPOWER**

| **Asajar's Work:** | |
|---|---|
| Theoretical Analysis | The ***brpower algorithm*** works faster than ***rpower*** because it ***reduces the value of n by about half during every recursive call***. For example, if n is 1,000, it can become 500, then 250, then 125, and continue until it reaches 0. Since the value is ***divided by two each time***, the number of calls grows ***much slower compared to rpower***. Because of this, the ***time complexity is O(log n)***. The space complexity is also ***O(log n)*** because the number of recursive calls is much smaller. |
| Experimental Analysis | The program tested ***brpower using input sizes of 1,000*** and ***10,000*** before the program reached the 100,000 test. For both 1,000 and 10,000, the ***execution time was 0 microseconds***. This does not mean that brpower literally takes no time to run. It means that the execution was ***very fast*** and the timer recorded it as ***0 microseconds***. The ***100,000 brpower test was not reached*** because ***rpower was executed first and caused a segmentation fault*** at that input size. |
| Analysis and Comparison | The experimental result supports the theoretical analysis of ***O(log n)***. For 1,000, ***rpower took 37 microseconds while brpower took 0 microseconds***. For 10,000, ***rpower took 280 microseconds while brpower again took 0 microseconds***. This shows that ***brpower was much faster in the given tests***. The reason is that ***brpower reduces n by about half in every recursive call***, while ***rpower only reduces it by one***. Because of this, ***brpower needs much fewer recursive calls*** and can handle bigger values of ***n more efficiently***. |

| **Godino's Work:** | |
|---|---|
| Theoretical Analysis | The brpower algorithm works differently from rpower because it divides the value of n by two during each recursive call. Instead of decreasing the value one by one, the problem becomes about half its previous size. For example, a value of 1,000 can become 500, then 250, then 125, and continue until it reaches 0. Because the value is reduced by half each time, the time complexity is O(log n). The space complexity is also O(log n) because the number of recursive calls grows much slower. |
| Experimental Analysis | The program tested brpower using input sizes of 1,000 and 10,000 in the provided output. For both input sizes, the execution time was recorded as 0 microseconds. This does not mean that the algorithm literally takes no time to execute. It means that the execution was very short and the measurement using microseconds resulted in 0. The output for the 100,000 input size was not completed because rpower caused a segmentation fault before brpower could be tested. |
| Analysis and Comparison | The experimental results support the theoretical analysis of O(log n). For an input size of 1,000, rpower took 37 microseconds, while brpower took 0 microseconds. For an input size of 10,000, rpower took 280 microseconds, while brpower again took 0 microseconds. This shows that brpower was much faster in the given tests. The reason is that brpower reduces the problem by about half in every recursive call, while rpower only reduces it by one. Because of this, brpower needs much fewer recursive calls and is more efficient for larger values of n. The 100,000 brpower test was not completed because rpower caused a segmentation fault before brpower could run. |

**Asajar's Conclusion:**

The activity gave a better understanding of how the performance of an algorithm can change depending on its input size and how the runtime can be checked using different types of analysis. For the lessons learned, the Unique algorithm uses nested loops to compare the elements, which gives it an $O(n^2)$ runtime, while rpower has O(n) because it decreases n one by one in every recursive call. The brpower algorithm has O(log n) because it reduces n by about half, making it faster than rpower. For the procedure, the codes were created and tested using different input sizes, then the execution times were recorded and compared with the expected runtime. The important parts such as Unique(), rpower(), brpower(), testUnique(), and testPower() helped show how each algorithm works and how the input size affects the execution time. For the supplementary activity, the runtime results were analyzed by comparing the theoretical complexity with the actual execution time from the program.

One difficulty faced was understanding how recursive calls work, especially when rpower reached an input size of 100,000 and caused a segmentation fault because of the many recursive calls. Another difficulty was understanding why brpower showed 0 microseconds for some inputs and why the 100,000 test was not reached.

Even with these difficulties, the results were recorded correctly and the experimental results generally matched the expected theoretical runtime. Overall, the activity was done well because the different runtimes were understood and the results were successfully analyzed, but there is still an area for improvement in understanding recursion, calculating time complexity, and comparing experimental results more carefully. More practice with different algorithms and runtime problems can help improve the understanding and confidence in analyzing algorithm performance.

**Godino's Conclusion:**

This activity helped in understanding how algorithms can be analyzed through theoretical and experimental analysis. It showed that different algorithms have different runtimes, such as $O(n^2)$ for the Unique algorithm, $O(n)$ for rpower, and $O(\log n)$ for brpower. The experimental results also showed that the Unique algorithm became much slower as the input size increased, while brpower was faster because it reduces the problem by half in every recursive call. The procedure and supplementary activity helped in understanding how input size affects the performance of an algorithm and why choosing an efficient algorithm is important. Overall, the activity was done well, but there are still areas for improvement, especially in calculating time complexity, understanding recursive algorithms, and interpreting experimental results. More practice with algorithm analysis and runtime testing would help improve understanding and accuracy in future activities.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **ChatGPT, Grammarly**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **formatting, explanation of concepts, grammar correction**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.